

化工熱力學（期末報告）

企業顧問報告：非理想混合系統之相平衡與能量分析

課程

Chemical Engineering Thermodynamics

指定理論基礎

Stanley I. Sandler

Chemical, Biochemical, and Engineering Thermodynamics

Chapters 6 - 10

一、顧問任務背景（Consulting Scenario）

你所屬的小組是受聘於一家精細化學品與生技材料公司的外部製程顧問團隊。該公司正計畫最適化一項涉及非理想液相混合與相平衡行為的單元操作（如蒸餾前混合槽、溶劑回收系統或萃取程序）。

公司過去使用理想溶液假設進行設計，導致：

- 能耗預估嚴重偏差
- 操作溫度不穩定
- 預期的相分離行為未發生

管理層要求你們基於嚴謹的熱力學分析（Sandler Ch. 6 - 10），重新評估該製程，並提出工程可行的修正建議。

二、指定系統（擇一）

建議系統

- Ethanol / Water

2. Acetone / Water
3. Ethanol / Ethyl Acetate
4. Acetone / Chloroform
5. Isopropanol / Water

系統需具有明顯非理想性，並可取得 NRTL 參數。

三、顧問報告任務內容 (ALL parts required)

Part I | 執行摘要 (Executive Summary) (10%)

(約 1 頁)

以高階主管可理解的語言說明：

- 本製程的核心熱力學問題
 - 為何理想溶液假設失效
 - 使用 NRTL 模型後，對設計決策造成的主要改變
 - 你們最重要的一項工程建議
-

Part II | 問題定義與理論背景 (20%)

1. 說明該單元操作（蒸餾／混合／萃取）的角色
2. 明確指出：
 - 涉及的相（VLE 或 LLE）
 - 為何需考慮 fugacity / activity coefficient
3. 寫出相平衡條件（不需完整推導）：

$$f_i(1)=f_i(2)f_i^{\{(1)\}} = f_i^{\{(2)\}}f_i(1)=f_i(2)$$

Part III | NRTL 模型與相平衡分析 (30%)

(A) NRTL 模型說明（質性）

- NRTL 的基本假設

- 參數 τ_{ij} , α_{ij} 的物理意義
- 為何 NRTL 適合此系統？

(B) 計算與比較 (定量)

1. 使用：
 - Raoult' s law (理想模型)
 - NRTL 模型
 2. 計算並比較：
 - 一組 T-x-y 或 P-x-y
 3. 至少提供：
 - 一張理想 vs NRTL 預測比較圖
 4. 說明：
 - 偏差最明顯的組成範圍
 - 若用理想模型，工程上會產生何種誤判？
-

Part IV | Excess Properties 與能量分析 (25%)

1. 使用 NRTL 計算：
 - G^{ex}
 - ΔH_{mix}
 2. 繪製至少一張圖：
 - G^{ex} vs mole fraction
 - ΔH_{mix} vs mole fraction
 3. 回答：
 - 混合為吸熱或放熱？
 - 對混合槽／溫控設計有何影響？
-

Part V | 顧問建議與工程判斷 (15%)

以顧問立場回應管理層：

- 此製程在目前條件下是否合理？
- 哪些操作變數最敏感 (T、P、組成) ？
- 你們建議：
 - 改變操作條件？

- 改用不同溶劑？
- 採用分段操作？

請明確說明建議背後的**熱力學理由**。

四、報告格式要求

- 小組人數：5~6 人
 - 頁數：6 - 10 頁（不含附錄）
 - 格式：企業顧問報告（非傳統作業）
 - 可使用工具：Excel / MATLAB / Python / Aspen
 - 所有模型與參數需註明來源
-

五、評分標準（供學生參考）

評分項目	比重
NRTL 模型理解與應用	30%
理想 vs 非理想比較分析	20%
Excess properties 與能量判斷	20%
工程建議合理性	20%
報告結構與表達	10%

Python Programming Code: (乙醇與水系統為例參考用)

```
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from typing import Callable, Tuple
from scipy.optimize import brentq

# =====
# Ethanol(1) - Water(2) Thermodynamics at 1 atm (101.3 kPa)
# Models: Raoult / Van Laar / NRTL
# Outputs (PNG):
#   (1) T-x-y diagram:   TxY_Ethanol_Water_1atm.png
#   (2) Y-x diagram:     Yx_Ethanol_Water_1atm.png
#   (3) GE vs x (NRTL): GE_vs_x_Ethanol_Water_1atm.png
#   (4) ΔHmix vs x (NRTL): Hmix_vs_x_Ethanol_Water_1atm.png
# =====

R = 8.314462618 # J/mol/K

# -----
# 1) Antoine vapor pressure
# -----
@dataclass(frozen=True)
class AntoineParams:
    A: float
    B: float
    C: float
    psat_unit: str = "mmHg"

def psat_antoine_kpa(T_C: float, p: AntoineParams) -> float:
    """Return Psat in kPa for temperature T_C (°C), using Antoine
    constants."""
    log10_psat = p.A - p.B / (T_C + p.C)
    Psat = 10.0 ** log10_psat
    return Psat * 0.133322 if p.psat_unit.lower() == "mmhg" else Psat

# -----
```

```

# 2) Activity coefficient models
# -----
def gamma_ideal(x1: float, T_K: float, params=None):
    """Raoult's law (ideal solution): gamma1 = gamma2 = 1."""
    return 1.0, 1.0

@dataclass(frozen=True)
class VanLaarParams:
    A12: float
    A21: float

def gamma_vanlaar(x1: float, T_K: float, p: VanLaarParams):
    """Van Laar activity coefficients for a binary mixture."""
    x2 = 1.0 - x1
    denom = (p.A12 * x1 + p.A21 * x2) ** 2
    denom = max(denom, 1e-30)
    ln_g1 = p.A12 * (p.A21 * x2) ** 2 / denom
    ln_g2 = p.A21 * (p.A12 * x1) ** 2 / denom
    return np.exp(ln_g1), np.exp(ln_g2)

@dataclass(frozen=True)
class NRTLParams:
    alpha12: float
    alpha21: float
    #  $\tau_{12}(T) = a_{12} + b_{12}/T$  ;  $\tau_{21}(T) = a_{21} + b_{21}/T$  (T in K)
    a12: float
    b12: float
    a21: float
    b21: float

def tau(a, b, T):
    return a + b / T

def gamma_nrtl(x1: float, T_K: float, p: NRTLParams):
    """NRTL activity coefficients for a binary mixture."""
    x2 = 1.0 - x1
    tau12 = tau(p.a12, p.b12, T_K)
    tau21 = tau(p.a21, p.b21, T_K)

```

```

G12 = np.exp(-p.alpha12 * tau12)
G21 = np.exp(-p.alpha21 * tau21)

D1 = max(x1 + x2 * G21, 1e-30)
D2 = max(x2 + x1 * G12, 1e-30)

ln_g1 = (x2**2) * (tau21 * (G21 / D1)**2 + (tau12 * G12) /
(D2**2))
ln_g2 = (x1**2) * (tau12 * (G12 / D2)**2 + (tau21 * G21) /
(D1**2))
return np.exp(ln_g1), np.exp(ln_g2)

# -----
# 3) Bubble-point VLE at fixed pressure
# -----
def bubble_residual(T_C, x1, P_kPa, psat1, psat2, gamma_model,
gamma_params):
    """Bubble condition: sum(x_i * gamma_i * Psat_i(T)) = P"""
    T_K = T_C + 273.15
    g1, g2 = gamma_model(x1, T_K, gamma_params)
    return (
        x1 * g1 * psat_antoine_kpa(T_C, psat1)
        + (1 - x1) * g2 * psat_antoine_kpa(T_C, psat2)
        - P_kPa
    )

def solve_bubble_T(x1, P_kPa, psat1, psat2, gamma_model,
gamma_params):
    """Solve bubble temperature at given x1 and P (kPa)."""
    return brentq(
        bubble_residual, 40.0, 120.0,
        args=(x1, P_kPa, psat1, psat2, gamma_model, gamma_params)
    )

def compute_Txy(P_kPa, psat1, psat2, gamma_model, gamma_params,
x_grid):
    """
    Compute bubble T (°C) and vapor composition y1 for each liquid

```

```

composition x1 at fixed P.
"""
T_list, y_list = [], []
for x1 in x_grid:
    T_C = solve_bubble_T(x1, P_kPa, psat1, psat2, gamma_model,
gamma_params)
    T_K = T_C + 273.15
    g1, g2 = gamma_model(x1, T_K, gamma_params)

    y1 = x1 * g1 * psat_antoine_kpa(T_C, psat1) / P_kPa
    y2 = (1 - x1) * g2 * psat_antoine_kpa(T_C, psat2) / P_kPa
    y1 /= (y1 + y2) # normalize

    T_list.append(T_C)
    y_list.append(y1)

return x_grid, np.array(y_list), np.array(T_list)

# -----
# 4) Excess properties from NRTL
# -----
def excess_gibbs_GE(x1, T_K, p):
    """Molar excess Gibbs energy:  $GE = RT \sum x_i \ln(\gamma_i)$ ."""
    g1, g2 = gamma_nrtl(x1, T_K, p)
    return R * T_K * (x1 * np.log(g1) + (1 - x1) * np.log(g2))

def excess_enthalpy_HE(x1, T_K, p, dT=1e-3):
    """
    Molar excess enthalpy ( $\approx \Delta H_{mix}$  when ideal mixing enthalpy
    neglected):
     $HE = -R T^2 \sum x_i (d \ln(\gamma_i)/dT)_x$ 
    """
    g1p, g2p = gamma_nrtl(x1, T_K + dT, p)
    g1m, g2m = gamma_nrtl(x1, T_K - dT, p)
    dln1 = (np.log(g1p) - np.log(g1m)) / (2 * dT)
    dln2 = (np.log(g2p) - np.log(g2m)) / (2 * dT)
    return -R * T_K**2 * (x1 * dln1 + (1 - x1) * dln2)

```

```

# -----
# 5) MAIN (1 atm version)
# -----
def main():
    P_kPa = 101.3 # 1 atm

    # Antoine constants (example set; ensure your selected constants
    are valid over the needed T range)
    ethanol = AntoineParams(8.20417, 1642.89, 230.3)
    water = AntoineParams(8.07131, 1730.63, 233.426)

    # Example Van Laar parameters (illustrative)
    vanlaar_p = VanLaarParams(1.65, 0.95)

    # Example NRTL parameters (illustrative)
    # NOTE: For meaningful  $\Delta H_{mix}$ , use literature parameters with
    temperature dependence (b12/b21 not zero) if available.
    nrtl_p = NRTLParams(
        alpha12=0.3, alpha21=0.3,
        a12=0.88, b12=0.0,
        a21=1.45, b21=0.0
    )

    x = np.linspace(0.01, 0.99, 60)

    # VLE: T-x-y for each model
    xi, yi, Ti = compute_Txy(P_kPa, ethanol, water, gamma_ideal,
    None, x)
    xv, yv, Tv = compute_Txy(P_kPa, ethanol, water, gamma_vanlaar,
    vanlaar_p, x)
    xn, yn, Tn = compute_Txy(P_kPa, ethanol, water, gamma_nrtl,
    nrtl_p, x)

    # ----- (1) T-x-y plot -----
    plt.figure()
    plt.plot(xi, Ti, label="Ideal (Raoult): T-x")
    plt.plot(xv, Tv, label="Van Laar: T-x")
    plt.plot(xn, Tn, label="NRTL: T-x")

```

```

plt.plot(yi, Ti, "--", label="Ideal (Raoult): T-y")
plt.plot(yv, Tv, "--", label="Van Laar: T-y")
plt.plot(yn, Tn, "--", label="NRTL: T-y")
plt.xlabel("Mole fraction of ethanol ( $x_1$  or  $y_1$ )")
plt.ylabel("Temperature (°C)")
plt.title("Ethanol - Water T-x-y Diagram at 1 atm (101.3 kPa)")
plt.legend()
plt.grid(True)
plt.savefig("TxY_Ethanol_Water_1atm.png", dpi=300)
plt.show()

# ----- (2) Y-x plot -----
plt.figure()
# reference line
plt.plot(x, x, ":", label="y = x (reference)")
# model predictions
plt.plot(xi, yi, label="Ideal (Raoult)")
plt.plot(xv, yv, label="Van Laar")
plt.plot(xn, yn, label="NRTL")
plt.xlabel("Liquid mole fraction of ethanol,  $x_1$ ")
plt.ylabel("Vapor mole fraction of ethanol,  $y_1$ ")
plt.title("Ethanol - Water Y - X Diagram at 1 atm (101.3 kPa)")
plt.legend()
plt.grid(True)
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.savefig("Yx_Ethanol_Water_1atm.png", dpi=300)
plt.show()

# ----- Excess properties (NRTL) evaluated along the bubble
line -----
TnK = Tn + 273.15
GE = np.array([excess_gibbs_GE(x1, T_K, nrtl_p) for x1, T_K in
zip(xn, TnK)])
HE = np.array([excess_enthalpy_HE(x1, T_K, nrtl_p) for x1, T_K in
zip(xn, TnK)])

# ----- (3) GE vs x -----

```

```

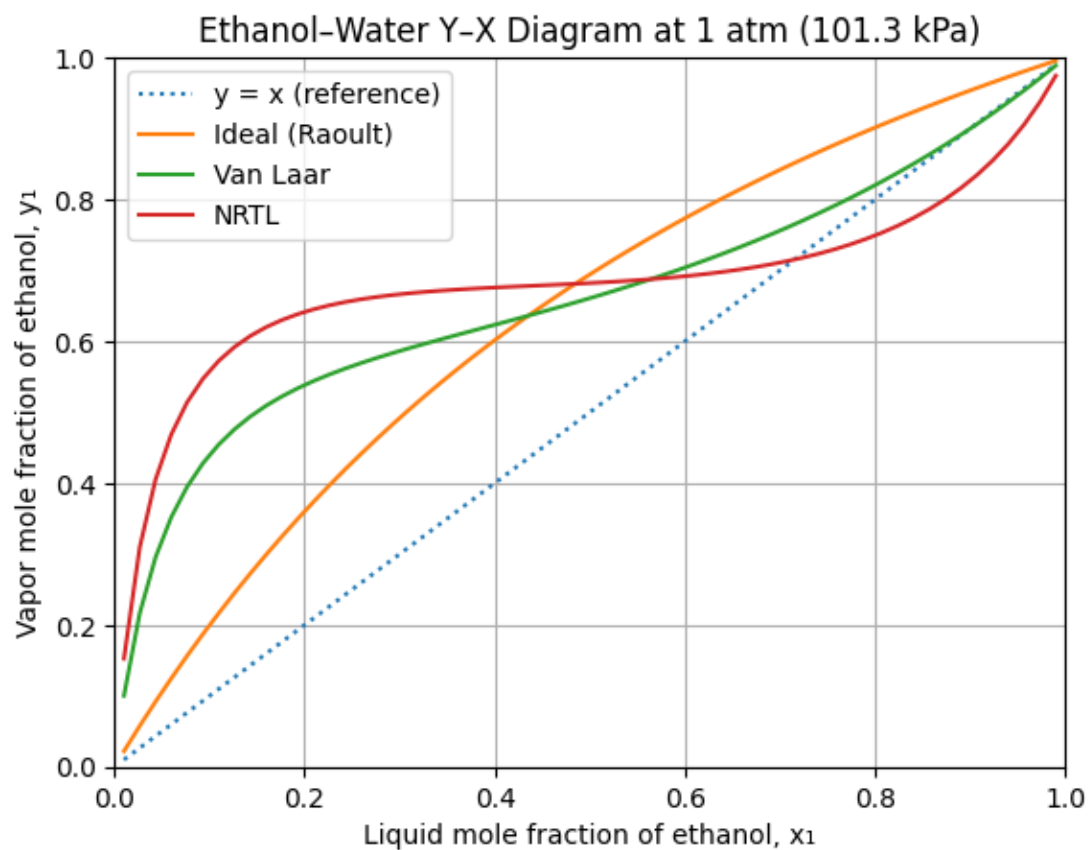
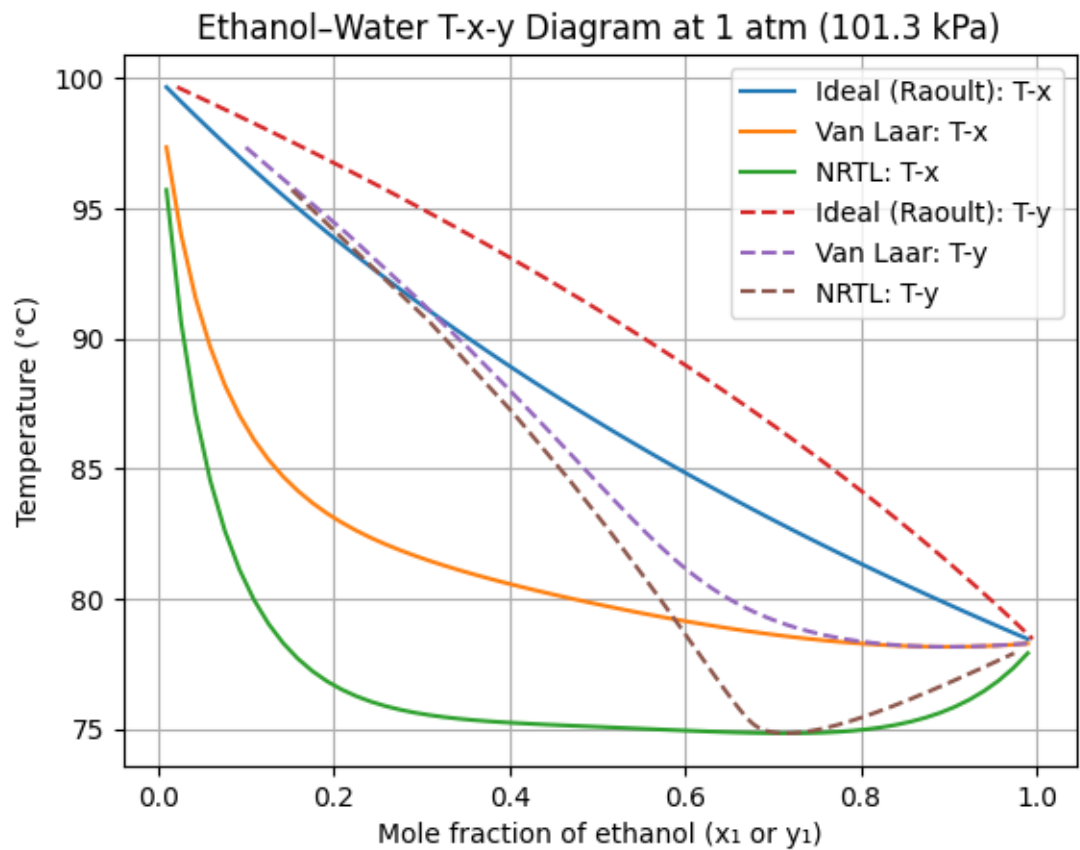
plt.figure()
plt.plot(xn, GE / 1000.0)
plt.xlabel("Liquid mole fraction of ethanol,  $x_1$ ")
plt.ylabel(" $G^{(E)}$  (kJ/mol)")
plt.title("NRTL Excess Gibbs Energy vs  $x_1$  at 1 atm (evaluated
along bubble line)")
plt.grid(True)
plt.savefig("GE_vs_x_Ethanol_Water_1atm.png", dpi=300)
plt.show()

# ----- (4)  $\Delta H_{mix}$  vs  $x$  -----
plt.figure()
plt.plot(xn, HE / 1000.0)
plt.xlabel("Liquid mole fraction of ethanol,  $x_1$ ")
plt.ylabel(" $\Delta H_{mix} \approx H^{(E)}$  (kJ/mol)")
plt.title("NRTL Excess Enthalpy vs  $x_1$  at 1 atm (evaluated along
bubble line)")
plt.grid(True)
plt.savefig("Hmix_vs_x_Ethanol_Water_1atm.png", dpi=300)
plt.show()

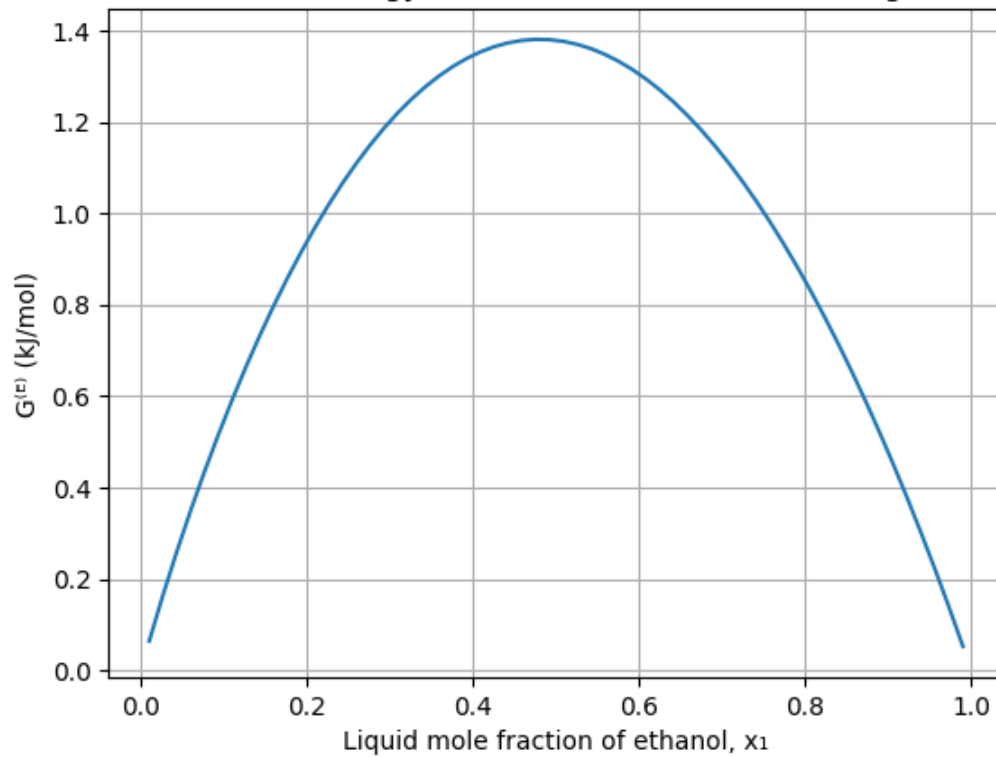
if __name__ == "__main__":
    main()

```

OUTCOME:



NRTL Excess Gibbs Energy vs x_1 at 1 atm (evaluated along bubble line)



NRTL Excess Enthalpy vs x_1 at 1 atm (evaluated along bubble line)

